

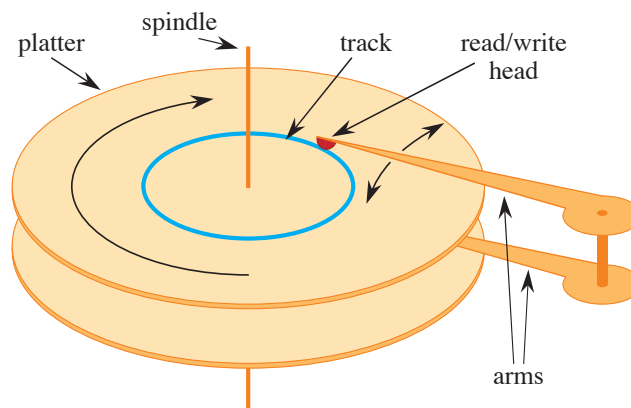


Figure 18.2 A typical magnetic disk drive. It consists of one or more platters covered with a magnetizable material (two platters are shown here) that rotate around a spindle. Each platter is read and written with a head, shown in red, at the end of an arm. Arms rotate around a common pivot axis. A track, drawn in blue, is the surface that passes beneath the read/write head when the head is stationary.

in drives for desktops, and 5400 RPM in drives for laptops. Although 7200 RPM may seem fast, one rotation takes 8.33 milliseconds, which is over 5 orders of magnitude longer than the 50 nanosecond access times (more or less) commonly found for main memory. In other words, if a computer waits a full rotation for a particular item to come under the read/write head, it could access main memory more than 100,000 times during that span. The average wait is only half a rotation, but still, the difference in access times for main memory compared with disk drives is enormous. Moving the arms also takes some time. As of this writing, average access times for commodity disk drives are around 4 milliseconds.

In order to amortize the time spent waiting for mechanical movements, also known as *latency*, disk drives access not just one item but several at a time. Information is divided into a number of equal-sized *blocks* of bits that appear consecutively within tracks, and each disk read or write is of one or more entire blocks.² Typical disk drives have block sizes running from 512 to 4096 bytes. Once the read/write head is positioned correctly and the platter has rotated to the beginning of the desired block, reading or writing a magnetic disk drive is entirely electronic (aside from the rotation of the platter), and the disk drive can quickly read or write large amounts of data.

² SSDs also exhibit greater latency than main memory and access data in blocks.

Often, accessing a block of information and reading it from a disk drive takes longer than processing all the information read. For this reason, in this chapter we'll look separately at the two principal components of the running time:

- the number of disk accesses, and
- the CPU (computing) time.

We measure the number of disk accesses in terms of the number of blocks of information that need to be read from or written to the disk drive. Although disk-access time is not constant—it depends on the distance between the current track and the desired track and also on the initial rotational position of the platters—the number of blocks read or written provides a good first-order approximation of the total time spent accessing the disk drive.

In a typical B-tree application, the amount of data handled is so large that all the data do not fit into main memory at once. The B-tree algorithms copy selected blocks from disk into main memory as needed and write back onto disk the blocks that have changed. B-tree algorithms keep only a constant number of blocks in main memory at any time, and thus the size of main memory does not limit the size of B-trees that can be handled.

B-tree procedures need to be able to read information from disk into main memory and write information from main memory to disk. Consider some object x . If x is currently in the computer's main memory, then the code can refer to the attributes of x as usual: $x.key$, for example. If x resides on disk, however, then the procedure must perform the operation `DISK-READ( $x$ )` to read the block containing object x into main memory before it can refer to x 's attributes. (Assume that if x is already in main memory, then `DISK-READ( $x$ )` requires no disk accesses: it is a “no-op.”) Similarly, procedures call `DISK-WRITE( $x$ )` to save any changes that have been made to the attributes of object x by writing to disk the block containing x . Thus, the typical pattern for working with an object is as follows:

```

 $x$  = a pointer to some object
DISK-READ( $x$ )
operations that access and/or modify the attributes of  $x$ 
DISK-WRITE( $x$ )           // omitted if no attributes of  $x$  were changed
other operations that access but do not modify attributes of  $x$ 

```

The system can keep only a limited number of blocks in main memory at any one time. Our B-tree algorithms assume that the system automatically flushes from main memory blocks that are no longer in use.

Since in most systems the running time of a B-tree algorithm depends primarily on the number of `DISK-READ` and `DISK-WRITE` operations it performs, we

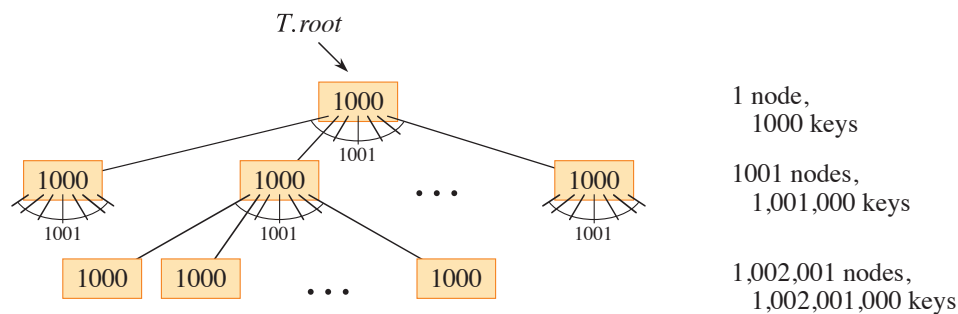


Figure 18.3 A B-tree of height 2 containing over one billion keys. Shown inside each node x is $x.n$, the number of keys in x . Each internal node and leaf contains 1000 keys. This B-tree has 1001 nodes at depth 1 and over one million leaves at depth 2.

typically want each of these operations to read or write as much information as possible. Thus, a B-tree node is usually as large as a whole disk block, and this size limits the number of children a B-tree node can have.

Large B-trees stored on disk drives often have branching factors between 50 and 2000, depending on the size of a key relative to the size of a block. A large branching factor dramatically reduces both the height of the tree and the number of disk accesses required to find any key. Figure 18.3 shows a B-tree with a branching factor of 1001 and height 2 that can store over one billion keys. Nevertheless, if the root node is kept permanently in main memory, at most two disk accesses suffice to find any key in this tree.

18.1 Definition of B-trees

To keep things simple, let's assume, as we have for binary search trees and red-black trees, that any satellite information associated with a key resides in the same node as the key. In practice, you might actually store with each key just a pointer to another disk block containing the satellite information for that key. The pseudocode in this chapter implicitly assumes that the satellite information associated with a key, or the pointer to such satellite information, travels with the key whenever the key is moved from node to node. A common variant on a B-tree, known as a *B⁺-tree*, stores all the satellite information in the leaves and stores only keys and child pointers in the internal nodes, thus maximizing the branching factor of the internal nodes.

A *B-tree* T is a rooted tree with root $T.root$ having the following properties:

1. Every node x has the following attributes:
 - a. $x.n$, the number of keys currently stored in node x ,
 - b. the $x.n$ keys themselves, $x.key_1, x.key_2, \dots, x.key_{x.n}$, stored in monotonically increasing order, so that $x.key_1 \leq x.key_2 \leq \dots \leq x.key_{x.n}$,
 - c. $x.leaf$, a boolean value that is TRUE if x is a leaf and FALSE if x is an internal node.
2. Each internal node x also contains $x.n + 1$ pointers $x.c_1, x.c_2, \dots, x.c_{x.n+1}$ to its children. Leaf nodes have no children, and so their c_i attributes are undefined.
3. The keys $x.key_i$ separate the ranges of keys stored in each subtree: if k_i is any key stored in the subtree with root $x.c_i$, then

$$k_1 \leq x.key_1 \leq k_2 \leq x.key_2 \leq \dots \leq x.key_{x.n} \leq k_{x.n+1}.$$

4. All leaves have the same depth, which is the tree's height h .
5. Nodes have lower and upper bounds on the number of keys they can contain, expressed in terms of a fixed integer $t \geq 2$ called the *minimum degree* of the B-tree:
 - a. Every node other than the root must have at least $t - 1$ keys. Every internal node other than the root thus has at least t children. If the tree is nonempty, the root must have at least one key.
 - b. Every node may contain at most $2t - 1$ keys. Therefore, an internal node may have at most $2t$ children. We say that a node is *full* if it contains exactly $2t - 1$ keys.³

The simplest B-tree occurs when $t = 2$. Every internal node then has either 2, 3, or 4 children, and it is a *2-3-4 tree*. In practice, however, much larger values of t yield B-trees with smaller height.

The height of a B-tree

The number of disk accesses required for most operations on a B-tree is proportional to the height of the B-tree. The following theorem bounds the worst-case height of a B-tree.

³ Another common variant on a B-tree, known as a *B*-tree*, requires each internal node to be at least $2/3$ full, rather than at least half full, as a B-tree requires.

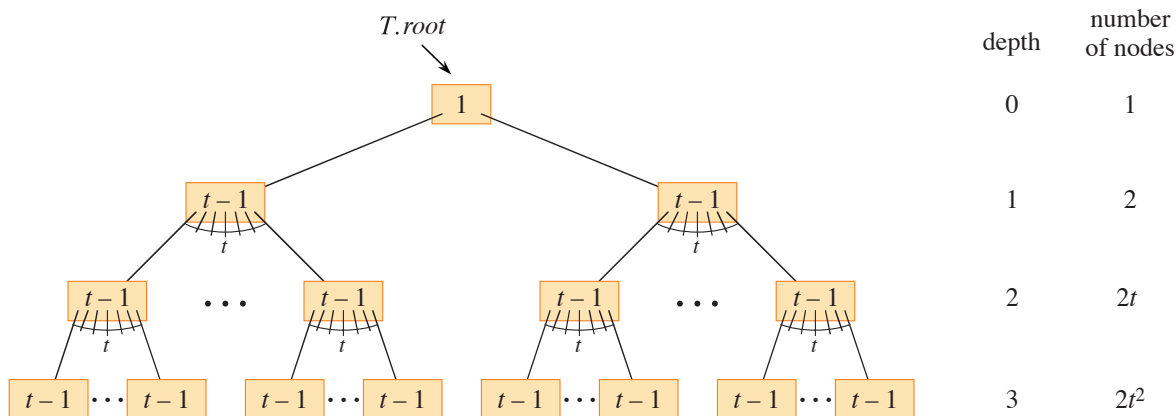


Figure 18.4 A B-tree of height 3 containing a minimum possible number of keys. Shown inside each node x is $x.n$.

Theorem 18.1

If $n \geq 1$, then for any n -key B-tree T of height h and minimum degree $t \geq 2$,

$$h \leq \log_t \frac{n+1}{2}.$$

Proof By definition, the root of a nonempty B-tree T contains at least one key, and all other nodes contain at least $t-1$ keys. Let h be the height of T . Then T contains at least 2 nodes at depth 1, at least $2t$ nodes at depth 2, at least $2t^2$ nodes at depth 3, and so on, until at depth h , it has at least $2t^{h-1}$ nodes. Figure 18.4 illustrates such a tree for $h = 3$. The number n of keys therefore satisfies the inequality

$$\begin{aligned} n &\geq 1 + (t-1) \sum_{i=1}^h 2t^{i-1} \\ &= 1 + 2(t-1) \left(\frac{t^h - 1}{t - 1} \right) \quad (\text{by equation (A.6) on page 1142}) \\ &= 2t^h - 1, \end{aligned}$$

so that $t^h \leq (n+1)/2$. Taking base- t logarithms of both sides proves the theorem. ■

You can see the power of B-trees as compared with red-black trees. Although the height of the tree grows as $O(\log n)$ in both cases (recall that t is a constant), for B-trees the base of the logarithm can be many times larger. Thus, B-trees save

a factor of about $\lg t$ over red-black trees in the number of nodes examined for most tree operations. Because examining an arbitrary node in a tree usually entails accessing the disk, B-trees avoid a substantial number of disk accesses.

Exercises

18.1-1

Why isn't a minimum degree of $t = 1$ allowed?

18.1-2

For what values of t is the tree of Figure 18.1 a legal B-tree?

18.1-3

Show all legal B-trees of minimum degree 2 that store the keys 1, 2, 3, 4, 5.

18.1-4

As a function of the minimum degree t , what is the maximum number of keys that can be stored in a B-tree of height h ?

18.1-5

Describe the data structure that results if each black node in a red-black tree absorbs its red children, incorporating their children with its own.

18.2 Basic operations on B-trees

This section presents the details of the operations B-TREE-SEARCH, B-TREE-CREATE, and B-TREE-INSERT. These procedures observe two conventions:

- The root of the B-tree is always in main memory, so that no procedure ever needs to perform a DISK-READ on the root. If any changes to the root node occur, however, then DISK-WRITE must be called on the root.
- Any nodes that are passed as parameters must already have had a DISK-READ operation performed on them.

The procedures are all “one-pass” algorithms that proceed downward from the root of the tree, without having to back up.

Searching a B-tree

Searching a B-tree is much like searching a binary search tree, except that instead of making a binary, or “two-way,” branching decision at each node, the search

makes a multiway branching decision according to the number of the node's children. More precisely, at each internal node x , the search makes an $(x.n + 1)$ -way branching decision.

The procedure B-TREE-SEARCH generalizes the TREE-SEARCH procedure defined for binary search trees on page 316. It takes as input a pointer to the root node x of a subtree and a key k to be searched for in that subtree. The top-level call is thus of the form B-TREE-SEARCH($T.root, k$). If k is in the B-tree, then B-TREE-SEARCH returns the ordered pair (y, i) consisting of a node y and an index i such that $y.key_i = k$. Otherwise, the procedure returns NIL.

```

B-TREE-SEARCH( $x, k$ )
1   $i = 1$ 
2  while  $i \leq x.n$  and  $k > x.key_i$ 
3       $i = i + 1$ 
4  if  $i \leq x.n$  and  $k == x.key_i$ 
5      return  $(x, i)$ 
6  elseif  $x.leaf$ 
7      return NIL
8  else DISK-READ( $x.c_i$ )
9      return B-TREE-SEARCH( $x.c_i, k$ )

```

Using a linear-search procedure, lines 1–3 of B-TREE-SEARCH find the smallest index i such that $k \leq x.key_i$, or else they set i to $x.n + 1$. Lines 4–5 check to see whether the search has discovered the key, returning if it has. Otherwise, if x is a leaf, then line 7 terminates the search unsuccessfully, and if x is an internal node, lines 8–9 recurse to search the appropriate subtree of x , after performing the necessary DISK-READ on that child. Figure 18.1 illustrates the operation of B-TREE-SEARCH. The blue nodes are those examined during a search for the key R .

As in the TREE-SEARCH procedure for binary search trees, the nodes encountered during the recursion form a simple path downward from the root of the tree. The B-TREE-SEARCH procedure therefore accesses $O(h) = O(\log_t n)$ disk blocks, where h is the height of the B-tree and n is the number of keys in the B-tree. Since $x.n < 2t$, the **while** loop of lines 2–3 takes $O(t)$ time within each node, and the total CPU time is $O(th) = O(t \log_t n)$.

Creating an empty B-tree

To build a B-tree T , first use the B-TREE-CREATE procedure on the next page to create an empty root node and then call the B-TREE-INSERT procedure on

page 508 to add new keys. Both of these procedures use an auxiliary procedure `ALLOCATE-NODE`, whose pseudocode we omit and which allocates one disk block to be used as a new node in $O(1)$ time. A node created by `ALLOCATE-NODE` requires no `DISK-READ`, since there is as yet no useful information stored on the disk for that node. `B-TREE-CREATE` requires $O(1)$ disk operations and $O(1)$ CPU time.

```

B-TREE-CREATE( $T$ )
1   $x = \text{ALLOCATE-NODE}()$ 
2   $x.\text{leaf} = \text{TRUE}$ 
3   $x.n = 0$ 
4  DISK-WRITE( $x$ )
5   $T.\text{root} = x$ 

```

Inserting a key into a B-tree

Inserting a key into a B-tree is significantly more complicated than inserting a key into a binary search tree. As with binary search trees, you search for the leaf position at which to insert the new key. With a B-tree, however, you cannot simply create a new leaf node and insert it, as the resulting tree would fail to be a valid B-tree. Instead, you insert the new key into an existing leaf node. Since you cannot insert a key into a leaf node that is full, you need an operation that *splits* a full node y (having $2t - 1$ keys) around its *median key* $y.\text{key}_t$ into two nodes having only $t - 1$ keys each. The median key moves up into y 's parent to identify the dividing point between the two new trees. But if y 's parent is also full, you must split it before you can insert the new key, and thus you could end up splitting full nodes all the way up the tree.

To avoid having to go back up the tree, just split every full node you encounter as you go down the tree. In this way, whenever you need to split a full node, you are assured that its parent is not full. Inserting a key into a B-tree then requires only a single pass down the tree from the root to a leaf.

Splitting a node in a B-tree

The procedure `B-TREE-SPLIT-CHILD` on the facing page takes as input a *nonfull* internal node x (assumed to reside in main memory) and an index i such that $x.c_i$ (also assumed to reside in main memory) is a *full* child of x . The procedure splits this child in two and adjusts x so that it has an additional child. To split a full root, you first need to make the root a child of a new empty root node, so that you can

use B-TREE-SPLIT-CHILD. The tree thus grows in height by 1: splitting is the only means by which the tree grows taller.

```

B-TREE-SPLIT-CHILD( $x, i$ )
1   $y = x.c_i$                                 // full node to split
2   $z = \text{ALLOCATE-NODE}()$                     //  $z$  will take half of  $y$ 
3   $z.\text{leaf} = y.\text{leaf}$ 
4   $z.n = t - 1$ 
5  for  $j = 1$  to  $t - 1$                         //  $z$  gets  $y$ 's greatest keys ...
6       $z.\text{key}_j = y.\text{key}_{j+t}$ 
7  if not  $y.\text{leaf}$ 
8      for  $j = 1$  to  $t$                         // ... and its corresponding children
9           $z.c_j = y.c_{j+t}$ 
10  $y.n = t - 1$                                 //  $y$  keeps  $t - 1$  keys
11 for  $j = x.n + 1$  downto  $i + 1$             // shift  $x$ 's children to the right ...
12      $x.c_{j+1} = x.c_j$ 
13  $x.c_{i+1} = z$                                 // ... to make room for  $z$  as a child
14 for  $j = x.n$  downto  $i$                     // shift the corresponding keys in  $x$ 
15      $x.\text{key}_{j+1} = x.\text{key}_j$ 
16  $x.\text{key}_i = y.\text{key}_t$                         // insert  $y$ 's median key
17  $x.n = x.n + 1$                                 //  $x$  has gained a child
18 DISK-WRITE( $y$ )
19 DISK-WRITE( $z$ )
20 DISK-WRITE( $x$ )

```

Figure 18.5 illustrates how a node splits. B-TREE-SPLIT-CHILD splits the full node $y = x.c_i$ about its median key (S in the figure), which moves up into y 's parent node x . Those keys in y that are greater than the median key move into a new node z , which becomes a new child of x .

B-TREE-SPLIT-CHILD works by straightforward cutting and pasting. Node x is the parent of the node y being split, which is x 's i th child (set in line 1). Node y originally has $2t$ children and $2t - 1$ keys, but splitting reduces y to t children and $t - 1$ keys. The t largest children and $t - 1$ keys of node y move over to node z , which becomes a new child of x , positioned just after y in x 's table of children. The median key of y moves up to become the key in node x that separates the pointers to nodes y and z .

Lines 2–9 create node z and give it the largest $t - 1$ keys and, if y and z are internal nodes, the corresponding t children of y . Line 10 adjusts the key count for y . Then, lines 11–17 shift keys and child pointers in x to the right in order to make room for x 's new child, insert z as a new child of x , move the median key

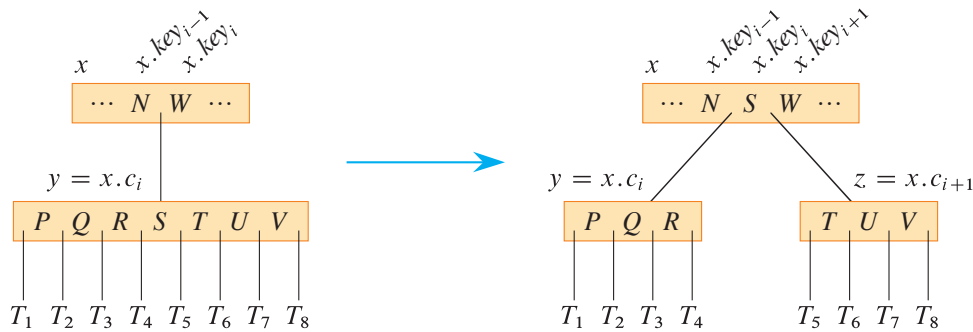


Figure 18.5 Splitting a node with $t = 4$. Node $y = x.c_i$ splits into two nodes, y and z , and the median key S of y moves up into y 's parent.

from y up to x in order to separate y from z , and adjust x 's key count. Lines 18–20 write out all modified disk blocks. The CPU time used by B-TREE-SPLIT-CHILD is $\Theta(t)$, due to the **for** loops in lines 5–6 and 8–9. (The **for** loops in lines 11–12 and 14–15 also run for $O(t)$ iterations.) The procedure performs $O(1)$ disk operations.

Inserting a key into a B-tree in a single pass down the tree

Inserting a key k into a B-tree T of height h requires just a single pass down the tree and $O(h)$ disk accesses. The CPU time required is $O(th) = O(t \log_t n)$. The B-TREE-INSERT procedure uses B-TREE-SPLIT-CHILD to guarantee that the recursion never descends to a full node. If the root is full, B-TREE-INSERT splits it by calling the procedure B-TREE-SPLIT-ROOT on the facing page.

```

B-TREE-INSERT( $T, k$ )
1   $r = T.root$ 
2  if  $r.n == 2t - 1$ 
3       $s = \text{B-TREE-SPLIT-ROOT}(T)$ 
4      B-TREE-INSERT-NONFULL( $s, k$ )
5  else B-TREE-INSERT-NONFULL( $r, k$ )

```

B-TREE-INSERT works as follows. If the root is full, then line 3 calls B-TREE-SPLIT-ROOT in line 3 to split it. A new node s (with two children) becomes the root and is returned by B-TREE-SPLIT-ROOT. Splitting the root, illustrated in Figure 18.6, is the only way to increase the height of a B-tree. Unlike a binary search tree, a B-tree increases in height at the top instead of at the bottom. Regardless of whether the root split, B-TREE-INSERT finishes by calling B-TREE-INSERT-NONFULL to insert key k into the tree rooted at the nonfull root node,